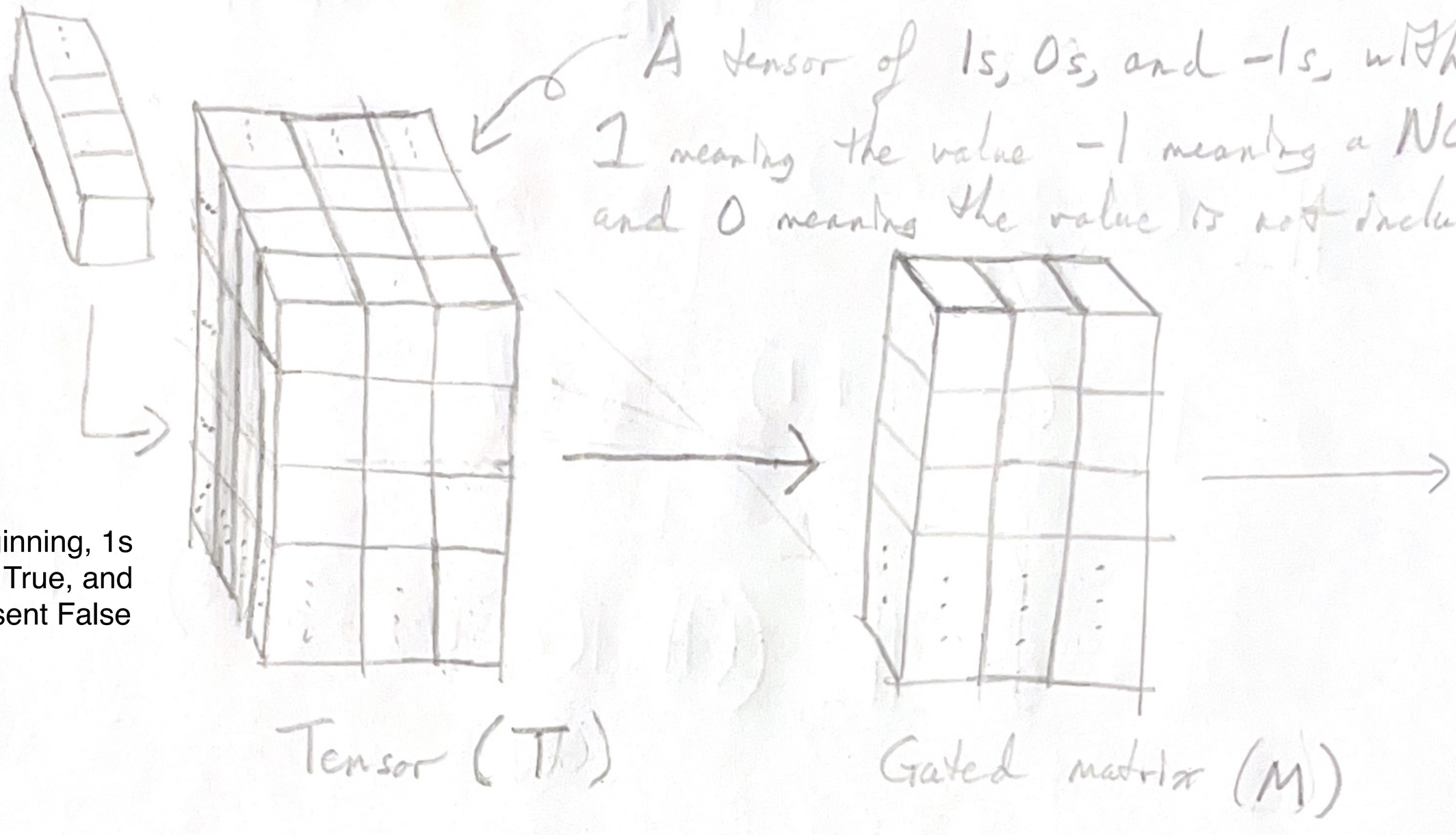
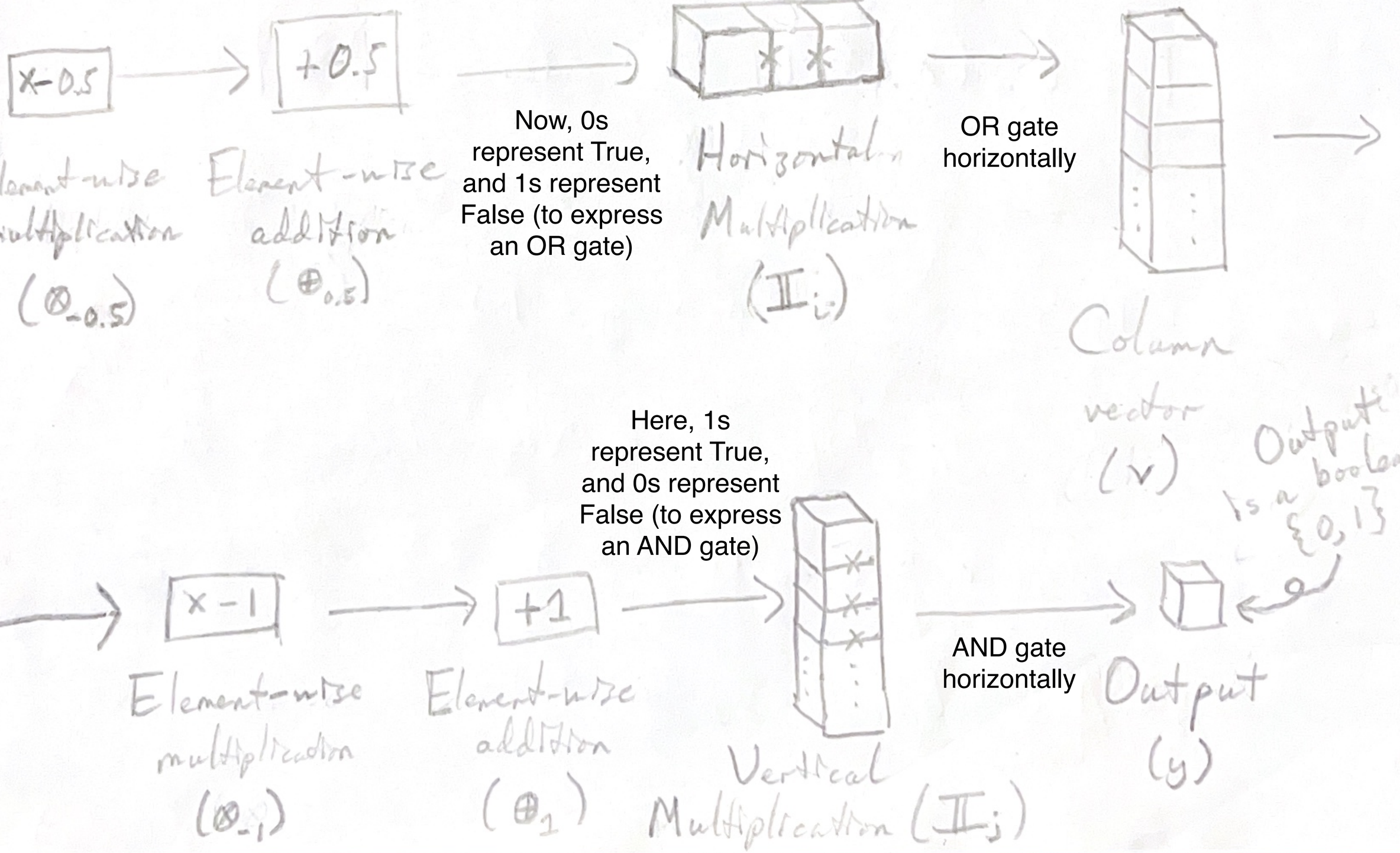
Simulating 3 SAT (algebraically)

Jalen Wang

Inputs (x) ↷ Inputs are 1 or -1 to match bool. T/F



In the beginning, 1s represent True, and -1s represent False



Algebraically:

$M = x \cdot T$ | $x \in \mathbb{R}^n$

$T \in \mathbb{R}^n$

$t \in \mathbb{R}^{m \times 3}$, for $t \in T$

$v = \coprod_i (M \cdot -0.5 + 0.5)_{ij}$ | $v \in \mathbb{R}^m$

$y = \coprod_j (v \cdot -1 + 1)_{ij}$ | $y \in \mathbb{R}$

So unfolded: $y = \coprod_j \left(\coprod_i (x \cdot T \cdot -0.5 + 0.5) \cdot -1 + 1 \right)$

Also, we want $x_0 \in \{-1, 1\}$ for $x_0 \in x$ or:

$$L_\infty(x) \leq 1 \quad \text{and} \quad L_{-\infty}(x) \geq 1$$

So the question is: Does there exist an x for which $x_0 \in \{-1, 1\}$ for $x_0 \in x$ and $y = 1$?

Apparently, *checking if the solutions to this system satisfy $x_0 \in \{-1, 1\}$ for $x_0 \in x$ is NP-complete.

* And solving the system first

$$M = x \cdot T$$

$$v = \prod_i \left(-0.5M + 0.5\right)_{ij}$$

$$y = \prod_j \left(-v + 1\right)_{ij}$$

$$y = 1$$

$$\langle 1, 1, 1, \ldots, 1 \rangle = \pi v + 1$$

$$v = \langle 0, 0, 0, \ldots, 0 \rangle$$

$$\langle 0, 0, 0, \ldots 0 \rangle = \prod_i \left(-0.5M + 0.5\right)_{ij}$$

$$\begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_m \end{bmatrix} = -0.5M + 0.5$$

$$A = \{ a_i \in \{ \langle 0,0,0 \rangle, \langle 0,0,1 \rangle, \langle 0,1,0 \rangle, \langle 0,1,1 \rangle, \langle 1,0,0 \rangle, \langle 1,0,1 \rangle, \langle 1,1,0 \rangle \} \}$$

$$\begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix} = M$$

$$B = \{ b_i \in \{ \langle 1,1,1 \rangle, \langle 1,1,-1 \rangle, \langle 1,-1,1 \rangle, \langle -1,1,1 \rangle, \langle 1,-1,-1 \rangle, \langle -1,1,-1 \rangle, \langle 1,1,-1 \rangle \} \}$$

$$\begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix} = \chi \cdot T$$

$$\begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix} = \begin{bmatrix} \chi \\ \vdots \\ \chi \end{bmatrix} \cdot \begin{bmatrix} t_0 \\ t_1 \\ t_2 \\ \vdots \\ t_m \end{bmatrix}$$

broadcast

$b_0 = \chi \cdot t_0$

vector (3)

matrix (n × 3)

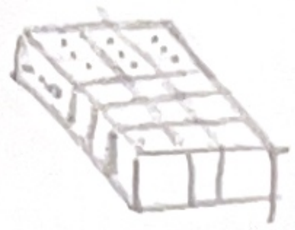
vector (n)

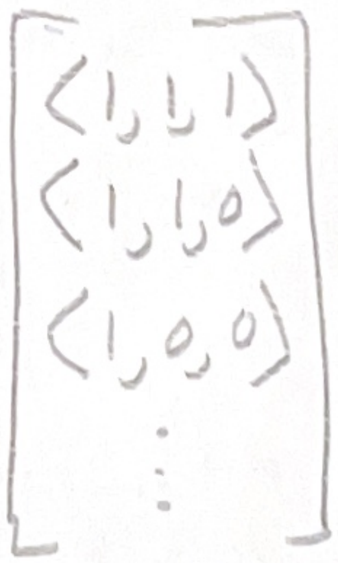


$$\begin{bmatrix} \langle 1, 1, 1 \rangle \\ \langle 1, 1, -1 \rangle \\ \langle 1, -1, 1 \rangle \\ \langle -1, 1, 1 \rangle \\ \langle 1, -1, -1 \rangle \\ \langle -1, 1, -1 \rangle \\ \langle -1, -1, 1 \rangle \\ \vdots \end{bmatrix}$$

$$\chi = b_0 \times t_0^{-1}$$
$$= b_1 \times t_1^{-1}$$
$$= b_2 \times t_2^{-1}$$
$$\vdots$$
$$= b_m \times t_m^{-1}$$

Calculating the number of configurations (not part of the solution)





with n=3:

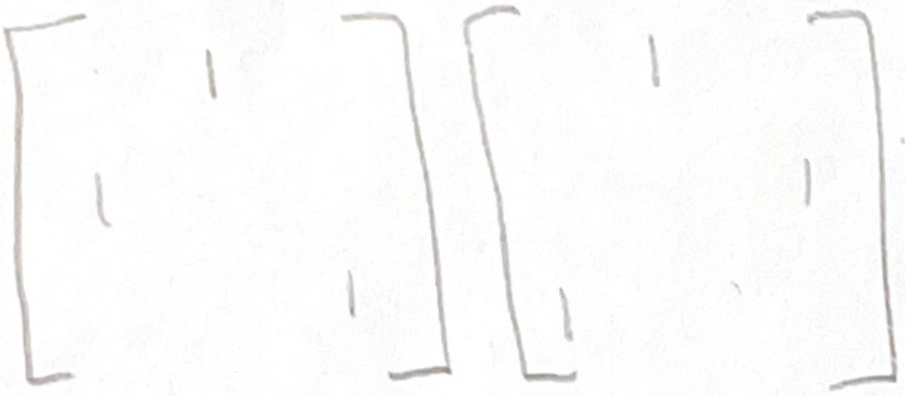
6 combos

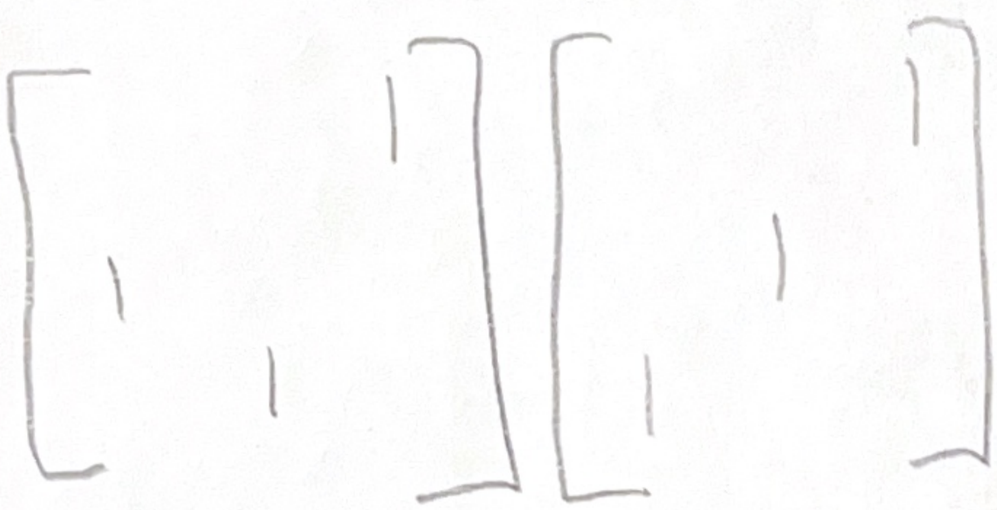




9 combos

6 combos x 9 combos = 54 combos





Let n_1 be the # of elements of n that are 1 and n_{-1} be the # of elements of n that are -1.

for $n_1=3$, $n_{-1}=0$:

learned 3 elements $\longleftarrow t_0^{-1}$

learns up to 3 elements $\longleftarrow t_1^{-1}$

for $n_1=2$, $n_{-1}=1$:

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & -1 \\ 1 & -1 & 1 \\ -1 & 1 & 1 \\ 1 & -1 & -1 \\ -1 & 1 & -1 \\ -1 & -1 & 1 \\ \vdots & \vdots & \vdots \end{bmatrix}$$

Each line confirms the value of 3 of the elements

B/c there are duplicates, each following line can learn ≤ 3 elements

Turns out, this is the same as...

t_0^{-1}
t_1^{-1}
t_2^{-1}
$\vdots$

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & -1 \\ 1 & -1 & -1 \\ \vdots & \vdots & \vdots \end{bmatrix}$$

Since all permutations can be expressed w/ T

→ There must exist some x s.t.:

Or, for each $t \in T$:

(This clause was written last, additionally, which is why it was written before "Or")

there must exist a $b \in B$ s.t.

$$b \cdot t^{-1} = x$$

Or,

There must exist some x s.t.

for each $t \in T$:

$$x \cdot t \neq \langle -1, -1, -1 \rangle$$

→ Wait no, that is actually where I was going for this.

For each $t \in T$:

there must exist some $b \in B$ s.t.

$b \cdot t^{-1} \cdot x$ does not contain -1

then:

for $\hat{x}_i$ in $b \cdot t^{-1}$:

$x_i = \hat{x}_i$ if $\hat{x}_i \neq 0$

let $X = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \rightarrow x_i \in \mathbb{R}$

---

let $X = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ \vdots \\ x_{2^n} \end{bmatrix} \rightarrow x_i \in \mathbb{R}^n$

$x_i = \text{binary_digits}(i)$

for each $t \in T$:

~~$b = np.array([-1,-1,-1])$~~

$X = np.ones((2)*n)$

for each $t \in T$:

$\vec{x} = X$

$\hat{x} = b \cdot t^{-1}$

for i, $\hat{x}_-$ in enumerate($\vec{x}$):

if $\hat{x}_-$:

$x = x[\vec{x} \div 2 - 1]$

else:

$x = x[:]$

```
X = False
if X.contains(True):
    continue
else:
    return False
return X.sum()
```